

Scikit-Learn (1.8.0) for Mechanical Engineering

A beginner-friendly tutorial on Python, machine learning, evaluation, workflows, supervised learning, unsupervised learning, and model interpretation

A self-contained beginner-friendly tutorial for Mechanical Engineering students learning Scikit-Learn for the first time

28 March 2026

This tutorial is written for Mechanical Engineering students who are new to Python programming, new to machine learning, and new to Scikit-Learn. It teaches the subject from first principles, keeps the mathematics accessible, and ties each major idea to Mechanical Engineering tasks such as material strength prediction, material classification, structural health monitoring, machined surface assessment, coolant quality classification, clustering of site observations, and anomaly detection in sensor streams [1][3][12][13].

This tutorial is organized into a teaching sequence that is easier for beginners to follow: Python basics, machine learning mounting bases, the standard Scikit-Learn workflow, evaluation and inspection tools, and then the major model families.

Table of Contents

1. Using This Tutorial
2. Learning Path for Beginners
3. Introduction to Scikit-Learn in Mechanical Engineering
4. Python Mounting bases Needed for This Tutorial
5. Machine Learning Mounting bases for Absolute Beginners
6. Core Scikit-Learn Workflow
7. Data Preparation, Splitting, Scaling, and Pipelines
8. Model Evaluation Tools and Mathematics
9. Model Inspection and Interpretability
10. Supervised Learning Models
11. Mathematical Background for Supervised Models
12. Unsupervised Learning Models
13. Mathematical Background for Unsupervised Models
14. Engineering Interpretation and Good Practice
15. Version Check and Setup
16. References

Major subsections inside these sections cover the main topic areas students typically need: classification metrics, regression metrics, clustering metrics, scorer objects, permutation importance, partial dependence, train and test splitting, scaling, pipelines, linear models, support vector machines, naive Bayes, decision trees, ensemble methods, clustering methods, dimensionality reduction, and anomaly detection.

1 Using This Tutorial

1.1 1.1 What this guide is trying to do

This guide has four goals.

First, it introduces Scikit-Learn as a practical Python library for machine learning. Second, it explains the meaning of the code patterns that appear again and again in Scikit-Learn, such as `fit`, `predict`, `transform`, and `score`. Third, it gives a beginner-friendly explanation of the main model families used throughout Scikit-Learn. Fourth, it keeps returning to Mechanical Engineering interpretation, because the point of learning the library is not only to run code, but also to make engineering judgments from data.

1.2 1.2 How to read the document

A good way to use the tutorial is:

- read Sections 3 to 7 in order if you are fully new to the topic;
- use Section 8 when you need to understand evaluation metrics;
- use Sections 10 and 12 as a reference when comparing models;
- use Sections 11 and 13 when you want the mathematics behind the models;
- return to Section 14 when you are deciding whether a result is useful enough for engineering work.

1.3 1.3 Notation used throughout the tutorial

The notation below will appear often.

Symbol or term	Meaning
X	The input data matrix. Each row is one sample and each column is one feature.
y	The target values or labels associated with the rows of X .
Sample	One observation, such as one material sample, one material sample, or one sensor time window.
Feature	A measured input variable, such as cement content, moisture content, or vibration amplitude.
Target	The output to be learned, such as compressive strength, class label, or anomaly flag.
Estimator	A Scikit-Learn object that learns from data. Most models and many preprocessing tools follow this interface.
Training set	The data used to fit the model.
Test set	The data held back until the end, used to check how the model performs on unseen data.

1.4 1.4 Code conventions used in the examples

All code examples in this guide are written in clean classroom style. They use fixed random seeds such as `random_state=42` so that students can reproduce the same result. They also use Scikit-Learn 1.8.0 conventions and avoid unnecessary complexity. When a short example uses synthetic data, the code is still teaching the real Scikit-Learn workflow that you would apply to real Mechanical Engineering data.

Section summary. This tutorial is a teaching guide, not a code dump. Read it as a sequence: ideas first, then code, then mathematics, then engineering interpretation.

2 Learning Path for Beginners

2.1 2.1 Recommended learning order

Scikit-Learn includes many ideas and tools, and beginners can easily feel that the topics arrive too quickly. A more effective production line for a first-time learner is the following:

1. Understand the absolute minimum Python needed to read the code.
2. Learn what machine learning is and what kinds of problems it solves.
3. Learn the standard Scikit-Learn pattern: define an estimator, fit it, predict or transform, and evaluate.
4. Learn how to split data, scale features, and protect against data leakage.
5. Learn the most important evaluation tools before learning many models.
6. Learn model inspection so you can ask what a model is using.
7. Learn supervised learning families.
8. Learn unsupervised learning families.
9. Return to the mathematics once the intuition is already in place.

That is the sequence used here.

2.2 2.2 Topic map for this tutorial

Table 2.1 summarizes how the major topics are organized in this tutorial.

Major topic	Where it appears in this tutorial	What this tutorial adds
Classification metrics	Section 8.1	Gentle mathematics, confusion matrix logic, Mechanical Engineering interpretation, cleaner code
Regression metrics	Section 8.2	Formula explanations, metric comparison, engineering interpretation
Clustering metrics	Section 8.3	Silhouette and adjusted Rand intuition, limitations, cleaner workflow
<code>make_scorer</code> and custom	Section 8.4	Why engineers may prefer custom loss ideas, simple

Major topic	Where it appears in this tutorial	What this tutorial adds
scoring		custom scorer example
Permutation importance	Section 9.2	Model-agnostic interpretation, limitations with correlated features
Partial dependence plots	Section 9.3	What the curves mean, when not to over-interpret them
train_test_split	Section 7.1	Why splitting matters and how to choose the test size
StandardScaler	Section 7.2	Scaling formula, why some models care more than others
make_pipeline and Pipeline	Section 7.3	Leakage prevention and consistent preprocessing
Linear and generalized linear models	Section 10.1	Full beginner explanation of linear regression, logistic regression, ridge, lasso, and elastic net
Support vector machines	Section 10.2	Margin intuition, scale sensitivity, classification and regression use
Naive Bayes	Section 10.3	Bayes rule intuition and differences among Gaussian, multinomial, and Bernoulli variants
Decision trees	Section 10.4	Split logic, strengths, weaknesses, engineering interpretation
Ensemble methods	Section 10.5	Random forests, boosting, extra trees, bagging, and model family comparison
Clustering algorithms	Section 12.1	Side-by-side conceptual comparison and Mechanical Engineering use cases
Dimensionality reduction	Section 12.2	PCA, t-SNE, LDA, NMF, and Isomap explained carefully
Anomaly detection	Section 12.3	Inlier and outlier logic, sensor-data examples

2.3 2.3 Why this structure works well for Mechanical Engineering students

Engineering students often ask three practical questions.

1. What does the model do?
2. What numbers go into it and come out of it?
3. When should I trust the result?

This document is structured around those questions. Every major topic starts with intuition, then moves to code, then gives just enough mathematics to make the idea precise.

Section summary. The tutorial covers a broad Scikit-Learn mounting base, taught in an order that is easier for an absolute beginner to follow.

3 Introduction to Scikit-Learn in Mechanical Engineering

3.1 3.1 What Scikit-Learn is

Scikit-Learn is a widely used open-source Python library for machine learning. It provides a large collection of algorithms and tools with a consistent interface, which means that many models are trained and used in almost the same way. This consistency is one of the main reasons the library is so popular in teaching and applied work [1][2][3].

In practice, Scikit-Learn sits inside the broader Python scientific computing ecosystem. A typical workflow uses:

- **NumPy** for arrays and numerical computation;
- **Pandas** for tabular data, if needed;
- **Matplotlib** for plots and diagnostics;
- **Scikit-Learn** for preprocessing, modeling, evaluation, and inspection.

3.2 3.2 What kinds of tasks Scikit-Learn supports

Scikit-Learn is useful for many common machine learning tasks [3][4]:

- **Regression**, where the target is a continuous number, such as mechanical material strength or predicted deflection.
- **Classification**, where the target is a class label, such as material type, coolant quality class, or machined surface condition category.
- **Clustering**, where the data are grouped without pre-assigned labels, such as grouping materials, site observations, or production throughput patterns.
- **Dimensionality reduction**, where many measured variables are compressed into a smaller number of informative directions.
- **Anomaly detection**, where unusual observations are flagged, such as abnormal vibration patterns in structural health monitoring.
- **Model selection and evaluation**, where models are compared fairly using metrics, cross-validation, and scoring tools.
- **Model inspection**, where the user studies which features matter and how a fitted model responds to them.

3.3 3.3 Why Mechanical Engineers should care

Mechanical Engineering increasingly uses data from laboratory testing, site investigations, inspection records, smart sensors, imaging systems, manufacturing networks, thermofluid monitoring, and cooling systems. Review studies show rapid growth of machine learning applications in structural health monitoring, vehicle systems, mechanics of materials and structures, environmental engineering, materials engineering, and mechanical systems management [12][13].

Typical Mechanical Engineering questions that can be studied with Scikit-Learn include:

- predicting mechanical material strength from mix proportions and curing age;
- classifying material or substrate condition from laboratory properties;
- identifying likely machined-surface defect classes from image or survey features;
- estimating deflection, deflection, or deformation from measured inputs;
- classifying coolant quality states from chemical indicators;
- grouping projects, materials, or site observations into similar clusters;
- reducing very high-dimensional sensor data into a few informative variables;
- detecting unusual structural or thermal-fluid behavior in monitoring streams.

3.4 3.4 Why Scikit-Learn is often a good starting point

For beginner-friendly engineering studies, Scikit-Learn has several advantages [2][3][7].

First, the interface is clean and stable. Second, it includes both simple baseline models and more powerful nonlinear models. Third, it supports preprocessing, evaluation, model selection, and inspection in one library. Fourth, its user guide is unusually strong, which matters when students need examples and trustworthy documentation.

3.5 3.5 What Scikit-Learn does not try to be

Scikit-Learn is excellent for classical machine learning on structured data. It is not primarily a deep learning library, and it is not usually the first choice for very large end-to-end neural network systems. In beginner Mechanical Engineering education, that is often an advantage rather than a weakness, because students can focus on data, models, assumptions, and evaluation without first needing advanced neural network tools.

Section summary. Scikit-Learn is a practical machine learning library that is especially useful when Mechanical Engineering problems can be written as structured prediction, classification, clustering, dimensionality reduction, or anomaly detection tasks.

4 Python Mounting bases Needed for This Tutorial

4.1 4.1 Intuition first

You do not need to be an experienced programmer to start using Scikit-Learn. You only need a small set of Python ideas. The important point is that Scikit-Learn code is object-oriented. In plain language, you create a model object, ask it to learn from data, and then ask it to make predictions or transformations.

4.2 4.2 Core Python concepts used in Scikit-Learn code

Python idea	Plain-language meaning	Example
Variable	A name that stores a value	<code>test_size = 0.2</code>

Python idea	Plain-language meaning	Example
Array	A structured collection of numbers	<code>X = np.array([[1.0, 2.0], [3.0, 4.0]])</code>
Function	A reusable action	<code>train_test_split(X, y)</code>
Class	A blueprint used to create objects	<code>LogisticRegression</code>
Object	A specific instance of a class	<code>model = LogisticRegression()</code>
Method	A function attached to an object	<code>model.fit(X, y)</code>
Argument	A value passed into a function or method	<code>test_size=0.2</code>
Keyword argument	An argument written as a name-value pair	<code>random_state=42</code>
Import	A way to bring external code into your script	<code>from sklearn.linear_model import LinearRegression</code>
Print	A way to display output	<code>print(model.score(X, y))</code>

4.3 4.3 NumPy arrays in beginner-friendly language

Scikit-Learn mainly works with numerical arrays. The most common data structure is a two-dimensional array called `X`.

- Each **row** of `X` is one sample.
- Each **column** of `X` is one feature.
- A separate one-dimensional array called `y` stores the target values.

For example, if you measured 50 material samples and recorded 4 input variables for each mix, then `X` would usually have shape `(50, 4)`. If you also recorded 50 compressive-strength values, then `y` would usually have shape `(50,)`.

4.4 4.4 A first code example

The short example below shows the smallest pattern that appears again and again in Scikit-Learn.

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Each row is one material sample.
# Columns: cement content, water content, curing age.
X = np.array([
    [320, 180, 7],
    [340, 175, 14],
    [360, 170, 28],
```



```

    [300, 190, 7],
    [380, 165, 28],
    [330, 178, 14],
], dtype=float)

# Target: compressive strength in MPa.
y = np.array([28.5, 33.0, 42.1, 24.7, 46.8, 35.2], dtype=float)

# Create an object from the LinearRegression class.
model = LinearRegression()

# Learn the relationship between X and y.
model.fit(X, y)

# Predict the strength of a new mix.
new_mix = np.array([[350, 172, 21]], dtype=float)
predicted_strength = model.predict(new_mix)

print("Predicted strength (MPa):", predicted_strength[0])

```

4.4.1 What the code is doing

- import numpy as np brings NumPy into the script and gives it the short name np.
- LinearRegression() creates a model object.
- model.fit(X, y) uses the training data to estimate the model parameters.
- model.predict(new_mix) returns a prediction for a new input row.

4.5 4.5 Classes, objects, and methods in one sentence

If you remember only one sentence from this section, remember this: **a Scikit-Learn class defines a type of model, an object is one specific model created from that class, and methods are the actions that object can perform.**

4.6 4.6 Plotting concepts, only as much as needed

Some Scikit-Learn tools create displays, such as confusion matrices or partial dependence plots. You do not need to be a plotting expert to follow the tutorial. At beginner level, it is enough to know that:

- a plot places values on axes so patterns can be seen quickly;
- Scikit-Learn can often create diagnostic displays directly;
- plots are useful for interpretation, but the main learning goal is to understand what the numbers mean.

4.7 4.7 Tabular data language

In engineering, the same words appear in many contexts. Here is the vocabulary used consistently in this guide.

Term	Meaning in this tutorial
Row	One sample or observation
Column	One measured variable
Feature	An input variable used by the model
Label	A class name, usually in classification
Target variable	The outcome to be predicted
Prediction	The model's estimated output for a sample

Section summary. The Python needed for Scikit-Learn is modest. You mainly need to understand arrays, imports, objects, methods, and the idea that X stores inputs while y stores outputs.

5 Machine Learning Mounting bases for Absolute Beginners

5.1 5.1 What machine learning means

Machine learning is the study of algorithms that learn patterns from data and use those patterns to make predictions, assign labels, find structure, or flag unusual cases [9][10]. Instead of writing a long list of explicit rules, we give the computer examples and ask it to learn a useful relationship.

In Mechanical Engineering, that relationship may link mix ingredients to strength, material indicators to class labels, or sensor measurements to a warning flag.

5.2 5.2 Supervised learning and unsupervised learning

The most important beginner distinction is whether labeled targets are available.

Learning type	What data you have	Typical question	Example
Supervised learning	Inputs X and known outputs y	Can we predict y from X ?	Predict material strength or classify material type
Unsupervised learning	Inputs X , but no known target y	What structure exists inside X ?	Group site observations or reduce many sensor variables to a few dimensions

5.3 5.3 Classification, regression, clustering, dimensionality reduction, and anomaly detection

- **Regression** predicts a number, such as deflection in millimeters.
- **Classification** predicts a class label, such as acceptable or unacceptable coolant quality.

- **Clustering** groups similar observations together without using known class labels.
- **Dimensionality reduction** compresses many variables into fewer informative variables.
- **Anomaly detection** tries to find samples that do not behave like the majority.

5.4 5.4 Features, labels, predictors, targets, and predictions

These terms are closely related and are often confused by beginners.

- **Features** are the measured inputs.
- **Predictors** is another name for input variables or features.
- **Labels** are class names in classification.
- **Target variables** are the outputs the model tries to learn.
- **Predictions** are the outputs produced by the trained model.

5.5 5.5 Training, testing, validation, and generalization

A model is useful only if it performs well on data it has not already seen. This idea is called **generalization**.

- The **training set** is used to fit the model.
- The **test set** is kept separate and used only after fitting.
- A **validation** process is used to compare settings or models during development. Cross-validation is a common validation strategy.

A model that memorizes the training data too closely may look excellent during training but fail on new data. This is called **overfitting**. A model that is too simple to capture the pattern is called **underfitting**.

5.6 5.6 Why data preparation matters

Machine learning is not only about choosing an algorithm. Data preparation matters because:

- missing or noisy data can distort the learned pattern;
- inconsistent units can confuse scale-sensitive models;
- badly split data can create unfair evaluations;
- leakage can make a model appear better than it really is;
- redundant or irrelevant features can reduce clarity and sometimes reduce performance.

For Mechanical Engineering data, preparation is especially important because measurements may come from different laboratories, instruments, units, or field conditions.

5.7 5.7 The general workflow of a Scikit-Learn project

A beginner-friendly project workflow is:

1. Define the engineering question.
2. Organize the data into features X and target y , if a target exists.
3. Split the data into training and testing sets.

4. Build a preprocessing and modeling workflow.
5. Fit the workflow on the training data.
6. Predict on the test data.
7. Evaluate the result with appropriate metrics.
8. Inspect the fitted model and interpret it in engineering terms.

5.8 5.8 A Mechanical Engineering mini-case

Suppose you want to predict mechanical material strength. You would collect features such as cement content, water content, supplementary cementitious material content, aggregate ratio, curing age, and admixture dosage. The target y would be measured compressive strength. This is a supervised regression problem. If, instead, you wanted to group similar material samples without using strength labels, that would be unsupervised clustering.

Section summary. Machine learning means learning useful patterns from data. For beginners, the key ideas are problem type, training versus testing, and the need to generalize beyond the examples already seen.

6 Core Scikit-Learn Workflow

6.1 6.1 The estimator interface

The heart of Scikit-Learn is the **estimator interface**. An estimator is any object that follows a standard pattern [3]. This is one of the most important ideas in the whole library, because it gives many tools the same structure.

Typical estimator methods are shown below.

Method	Plain-language meaning	Common use
<code>fit(X, y)</code>	Learn from the data	Train a model or estimate a transformation
<code>predict(X)</code>	Produce predicted outputs	Classification or regression
<code>score(X, y)</code>	Return a default performance score	Quick baseline evaluation
<code>transform(X)</code>	Convert data into a new representation	Scaling, PCA, feature engineering
<code>fit_transform(X, y=None)</code>	Learn a transformation and apply it in one step	StandardScaler, PCA, NMF
<code>predict_proba(X)</code>	Return class probabilities	Probabilistic classification, especially logistic regression, naive Bayes, random forests

6.2 6.2 Transformers and predictive models

A **transformer** changes the representation of data. Examples include StandardScaler, PCA, and NMF.

A **predictive model** learns a mapping from inputs to outputs. Examples include LinearRegression, LogisticRegression, RandomForestClassifier, and SVR.

Some workflows use both. For example, scaling may come first, followed by a classifier.

6.3 6.3 Why the same structure matters

Because many tools share the same interface, you can change one model for another with only small code changes. This makes experiments cleaner, reduces beginner confusion, and supports fair comparison across methods [3][6].

6.4 6.4 A minimal standard pattern

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# X has shape (n_samples, n_features).
# y has shape (n_samples,).
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000, random_state=42)
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
score = model.score(X_test, y_test)
```

6.4.1 Inputs and outputs in this pattern

- `X_train` and `X_test` are input matrices.
- `y_train` and `y_test` are target arrays.
- `model.fit(...)` uses the training set only.
- `model.predict(...)` returns predicted class labels.
- `model.predict_proba(...)` returns class probabilities. For binary classification, the second column usually represents the probability of class 1.
- `model.score(...)` returns a default score. For classifiers, this is often accuracy. For many regressors, it is the coefficient of determination, R^2 .

6.5 6.5 Why beginners should care about workflow structure

Students often focus on the algorithm name and ignore the workflow. In real work, a clean workflow is just as important as the algorithm because it controls reproducibility, fairness of evaluation, and prevention of mistakes. In engineering, a well-organized workflow also makes it easier to explain your method to a supervisor, reviewer, or design team.

Section summary. Scikit-Learn becomes much easier once you understand the estimator interface. Most of the library can be learned as variations on the same small pattern: split, fit, predict or transform, and evaluate.

7 Data Preparation, Splitting, Scaling, and Pipelines

7.1 7.1 train_test_split

7.1.1 Intuition

If you evaluate a model on the same data it saw during training, the result is too optimistic. `train_test_split` solves the first version of that problem by separating data into a training part and a testing part [4].

7.1.2 What it does

`train_test_split` randomly divides arrays or matrices into training and testing subsets.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y # useful for classification
)
```

7.1.3 Inputs and outputs

- **Inputs:** one or more arrays with matching row counts.
- **Outputs:** the training and testing portions of those arrays.

7.1.4 Beginner note

For classification, `stratify=y` helps preserve class proportions in both splits. This is especially useful when one class is rare.

7.2 7.2 StandardScaler

7.2.1 Intuition

Many algorithms care about the scale of the input variables. If one feature is measured in very large units and another in very small units, the large-scale feature may dominate the learning process even if it is not actually more important [5].

7.2.2 What it does

StandardScaler subtracts the training mean and divides by the training standard deviation so that each feature has approximately zero mean and unit standard deviation.

$$z = \frac{x - \bar{x}}{s}$$

Here, x is an original feature value, $\bar{x}$ is the training-set mean for that feature, and s is the training-set standard deviation.

7.2.3 Why scaling matters in Mechanical Engineering

Consider a dataset with:

- curing age in days,
- cement content in kilograms per cubic meter,
- water-cement ratio,
- ultrasonic pulse velocity in meters per second.

These variables live on very different numerical scales. Scale-sensitive models, such as support vector machines, logistic regression with regularization, k -means clustering, and principal component analysis, often behave better after scaling [3][5].

7.3 7.3 make_pipeline and Pipeline

7.3.1 Intuition

A pipeline chains steps together so that preprocessing and modeling happen in a controlled order [6]. This matters because the same transformation learned from the training data must be applied to future data in the same way.

7.3.2 What a pipeline protects you from

A common beginner mistake is to scale the full dataset before splitting it. That leaks information from the test set into the training procedure. Scikit-Learn's guidance explicitly warns against this kind of inconsistent preprocessing and data leakage [7].

7.3.3 Clean example

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

# Example features: plasticity index, water content, dry density, fines content.
X = np.array([
    [18.0, 12.5, 1.89, 16.0],
    [20.5, 14.1, 1.84, 19.0],
    [11.0, 8.2, 2.02, 9.0],
    [24.0, 17.2, 1.75, 26.0],
    [13.5, 9.6, 1.98, 11.5],
    [26.0, 18.8, 1.70, 28.0],
    [12.4, 8.9, 2.05, 10.0],
    [22.8, 15.4, 1.80, 23.0],
    [10.6, 7.9, 2.06, 8.0],
    [25.3, 17.9, 1.73, 27.0],
], dtype=float)

# 1 = weak substrate, 0 = acceptable substrate.
y = np.array([0, 0, 0, 1, 0, 1, 0, 1, 0, 1], dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", probability=True, random_state=42)
)

model.fit(X_train, y_train)
print("Test accuracy:", model.score(X_test, y_test))
```

7.3.4 Why each major line is present

- `train_test_split(...)` keeps testing fair.
- `StandardScaler()` standardizes the features inside the workflow.
- `SVC(...)` defines the predictive model.
- `make_pipeline(...)` ensures the same learned scaling is applied to both training and testing data.

7.4 7.4 Data leakage in simple terms

Data leakage means that information from outside the training set, often from the test set, leaks into the training process. Leakage makes a model look better than it really is. It is one of the most common reasons beginner results are unrealistically strong.

Typical leakage examples include:

- scaling all data before splitting;
- selecting features using the full dataset before splitting;
- cleaning missing values using test-set information;
- choosing model settings after repeatedly looking at the test result.

7.5 7.5 Which models are sensitive to scale

More sensitive to scale	Less sensitive to scale
Logistic regression	Decision trees
Support vector classifier and regressor	Random forests
k-means clustering	Extra trees
Principal component analysis	Gradient boosting trees
One-class SVM	Bagging with tree base learners

Tree-based models split on ordered thresholds, so raw scale is often less important. Distance-based and geometry-based models usually care much more.

Section summary. Good machine learning practice begins before model fitting. Split data fairly, scale when needed, place preprocessing inside a pipeline, and protect the test set from leakage.

8 Model Evaluation Tools and Mathematics

8.1 8.1 Classification metrics

8.1.1 Intuition

In classification, a model decides which class each sample belongs to. One metric is rarely enough. A model may have high overall accuracy but still miss important positive cases, which can be dangerous in engineering screening problems.

Suppose class 1 means “weak substrate” or “unsafe condition detected.” Then missing a true class 1 case can be more serious than raising a false alarm.

8.1.2 Confusion matrix structure

For binary classification, the confusion matrix is built from four counts:

Actual / Predicted	Predicted 0	Predicted 1
Actual 0	True negative (TN)	False positive (FP)
Actual 1	False negative (FN)	True positive (TP)

This table is the basis of accuracy, precision, recall, and the F1-score.

8.1.3 Accuracy score

Accuracy is the proportion of all predictions that are correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Purpose:** quick overall correctness.
- **Useful when:** the classes are reasonably balanced and both error types matter similarly.
- **Limitation:** accuracy can be misleading for imbalanced data.

8.1.4 Precision score

Precision asks: among the samples predicted as positive, how many were truly positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Purpose:** control false positives.
- **Useful when:** false alarms are costly.
- **Mechanical example:** if class 1 means “repair needed,” low precision means many unnecessary inspections or interventions.

8.1.5 Recall score

Recall asks: among the truly positive samples, how many did the model find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Purpose:** control false negatives.
- **Useful when:** missing a positive case is costly.
- **Mechanical example:** in screening for possible structural damage, low recall means missed warning cases.

8.1.6 F1-score

The F1-score balances precision and recall using the harmonic mean.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Purpose:** summarize the balance between precision and recall.

- **Useful when:** both false positives and false negatives matter.
- **Limitation:** it does not use true negatives directly.

8.1.7 ROC AUC score

The receiver operating characteristic (ROC) curve plots the true positive rate against the false positive rate over many thresholds. The area under this curve, called ROC AUC, summarizes how well the model separates the two classes.

A simple intuition is this: if you choose one positive and one negative sample at random, the ROC AUC is the probability that the model ranks the positive sample as more likely to be positive than the negative sample.

- **Purpose:** threshold-independent ranking quality.
- **Useful when:** you want to compare classifiers beyond a single cutoff.
- **Limitation:** it can hide threshold-specific consequences that matter in engineering decisions.

8.1.8 Classification report

A classification report combines precision, recall, F1-score, and support in one table. **Support** means the number of true samples in each class.

8.1.9 Worked example, binary material-condition classification

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
)

# Columns: liquid limit, plasticity index, water content, dry density.
X = np.array([
    [42, 19, 16.2, 1.83], [38, 16, 13.8, 1.90],
    [47, 24, 18.7, 1.76], [35, 12, 11.4, 1.97],
    [50, 27, 20.1, 1.72], [33, 11, 10.8, 2.01],
    [46, 23, 17.9, 1.79], [36, 13, 12.3, 1.95],
    [48, 25, 19.3, 1.74], [34, 10, 10.6, 2.03],
    [44, 21, 16.9, 1.81], [37, 14, 12.9, 1.92],
```

```

], dtype=float)

# 1 = weak substrate, 0 = acceptable substrate.
y = np.array([1, 0, 1, 0, 1, 0, 1, 0, 1, 0], dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000, random_state=42)
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification report:\n", classification_report(y_test, y_pred))

```

If you also want a professional confusion matrix display in Scikit-Learn 1.8.0, you can use the display API directly:

```

from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)

```

8.1.10 Mechanical Engineering interpretation

If this model is used as a screening tool for weak substrate, recall may matter more than accuracy. Missing a weak zone may be riskier than sending an extra sample for inspection. In that case, a model with slightly lower accuracy but higher recall could be the better engineering choice.

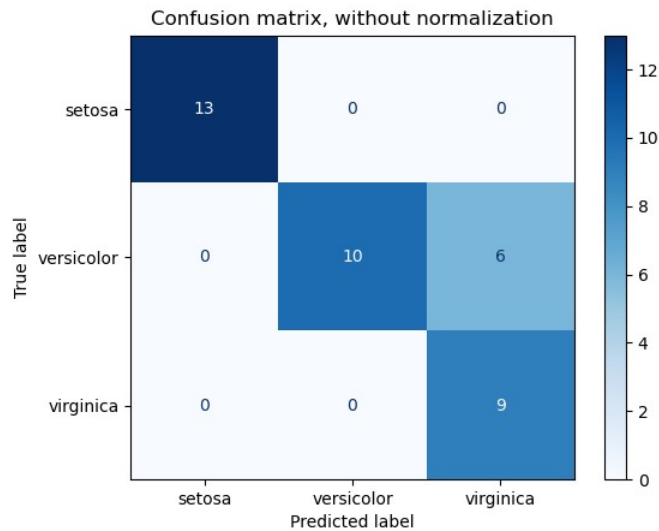


Figure 1. Example confusion matrix display from the Scikit-Learn example gallery. Source: Scikit-Learn developers, “Confusion matrix” example [14].

8.1.11 Classification metrics at a glance

Metric	What it tells you	Strong when	Weak when
Accuracy	Overall proportion correct	Classes are balanced	One class is rare
Precision	Quality of positive predictions	False positives are costly	False negatives matter more
Recall	How many true positives were found	Missing positives is costly	Too many false positives are tolerated
F1-score	Balance of precision and recall	Both errors matter	True negatives also matter strongly
ROC AUC	Ranking quality across thresholds	Comparing binary classifiers	Decision must use one specific threshold

8.2 Regression metrics

8.2.1 Intuition

Regression predicts a number, not a class. We therefore measure error by comparing predicted values to true values.

8.2.2 Mean squared error

The mean squared error (MSE) averages squared prediction errors.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Purpose:** penalize large errors strongly.
- **Why squaring matters:** large errors become much larger after squaring.
- **Limitation:** sensitive to outliers.

8.2.3 Mean absolute error

The mean absolute error (MAE) averages absolute prediction errors.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Purpose:** average error magnitude in the original units.
- **Why absolute value matters:** it measures distance from truth without cancellation.
- **Limitation:** it does not penalize very large errors as strongly as MSE.

8.2.4 R^2 score

The coefficient of determination, called R^2 , measures how much of the variation in the target is explained by the model.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Here, $\bar{y}$ is the mean of the true target values.

- **Purpose:** compare a model to the simple baseline of always predicting the mean.
- **Interpretation:** closer to 1 is better; 0 means no improvement over the mean baseline.
- **Limitation:** a high R^2 does not guarantee small engineering error in absolute units.

8.2.5 Explained variance score

Explained variance measures how much variability in the target is captured by the predictions.

$$\text{Explained Variance} = 1 - \frac{\text{Var}(y - \hat{y})}{\text{Var}(y)}$$

- **Purpose:** quantify how much target variation is accounted for.
- **Interpretation:** closer to 1 is better.
- **Limitation:** it can behave differently from R^2 when prediction bias is present.

8.2.6 Worked example, material strength prediction

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_squared_error,
```

```

mean_absolute_error,
r2_score,
explained_variance_score,
)

# Columns: cement, water, age (days).
X = np.array([
    [320, 180, 7], [320, 180, 28], [340, 175, 14],
    [340, 175, 28], [360, 170, 14], [360, 170, 28],
    [300, 190, 7], [300, 190, 28], [380, 165, 14],
    [380, 165, 28], [330, 178, 14], [350, 172, 21],
], dtype=float)

y = np.array([28.5, 37.8, 32.4, 40.9, 35.6, 44.1,
              24.7, 34.2, 38.8, 46.8, 33.7, 39.5], dtype=float)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R^2:", r2_score(y_test, y_pred))
print("Explained variance:", explained_variance_score(y_test, y_pred))

```

8.2.7 Mechanical Engineering interpretation

If the MAE is 1.8 MPa, that means the average absolute error is about 1.8 MPa in the same units engineers already use. This is why MAE is often easy to explain to non-specialists. MSE can still be useful because it penalizes rare large mistakes more heavily.

8.2.8 Regression metrics at a glance

Metric	Main meaning	Units	Good for
MSE	Average squared error	Squared target units	Penalizing large mistakes
MAE	Average absolute error	Same as target	Easy practical interpretation
R^2	Proportion of variation explained	Unitless	Comparing models on same dataset
Explained variance	Captured variability	Unitless	Understanding variability captured

8.3 Clustering metrics

8.3.1 Silhouette score

The silhouette score measures how well a sample matches its own cluster compared with other clusters.

For one sample i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where:

- $a(i)$ is the average distance from sample i to points in its own cluster,
- $b(i)$ is the smallest average distance from sample i to another cluster.

The overall silhouette score is the mean of these sample scores.

- values near 1 suggest well-separated clusters;
- values near 0 suggest overlapping clusters;
- negative values suggest possible misassignment.

8.3.2 Adjusted Rand score

The adjusted Rand score compares two cluster labelings. It is useful when true group labels are known and you want to compare the predicted clustering to that reference. It adjusts for agreement that could occur by chance.

- **Purpose:** compare cluster assignments to known labels.
- **Useful when:** you have a benchmark labeling.
- **Limitation:** in many real unsupervised problems, true labels do not exist.

8.3.3 Worked example, clustering site observations

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

# Imagine three groups of site observations: dense sand, silty material, soft clay.
X = np.array([
    [1.1, 1.0], [1.2, 1.1], [0.9, 1.2],
    [5.1, 5.0], [5.2, 4.9], [4.8, 5.2],
    [8.8, 1.2], [9.1, 0.9], [8.9, 1.0],
], dtype=float)

# Optional reference labels for teaching only.
y_true = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2])
```



```

model = KMeans(n_clusters=3, n_init="auto", random_state=42)
labels = model.fit_predict(X)

print("Silhouette score:", silhouette_score(X, labels))
print("Adjusted Rand score:", adjusted_rand_score(y_true, labels))

```

8.3.4 Mechanical Engineering interpretation

A high silhouette score suggests that the discovered groups are compact and well separated. In a materials study, that could indicate that the measured properties naturally form meaningful groups. The adjusted Rand score is useful in a classroom or benchmark study where the true material type is already known.

8.4 8.4 make_scorer and custom scoring function concepts

8.4.1 Why custom scoring exists

Built-in metrics are useful, but engineering problems sometimes have uneven error costs. For example, missing a critical damage case may be worse than creating an extra inspection visit. In such situations, you may want a custom score.

`make_scorer` converts a Python function into a scorer object that can be used in model evaluation tools such as `cross_val_score`, `cross_validate`, `GridSearchCV`, and `RandomizedSearchCV` [4].

8.4.2 Simple conceptual example

```

import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score, make_scorer
from sklearn.model_selection import cross_val_score

X, y = make_classification(
    n_samples=200,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42,
)

# Example custom score: weight recall strongly.
def weighted_recall(y_true, y_pred):
    return 2.0 * recall_score(y_true, y_pred)

recall_priority_scorer = make_scorer(weighted_recall)
model = LogisticRegression(max_iter=1000, random_state=42)

```

```
scores = cross_val_score(model, X, y, cv=5, scoring=recall_priority_scorer)
print("Fold scores:", scores)
print("Mean score:", np.mean(scores))
```

8.4.3 What the inputs and outputs mean

- A scoring function usually receives `y_true` and `y_pred`.
- It returns a single number, where larger is better unless you intentionally define it differently.
- `make_scorer(...)` wraps that function so Scikit-Learn can call it inside evaluation tools.

8.5 8.5 What metric choice tells you

Metrics do not answer the same question. A responsible engineering workflow therefore chooses metrics based on the cost of error, not only on convenience.

- use **recall** when missing positives is dangerous;
- use **precision** when false alarms are expensive;
- use **MAE** when you need error in physical units;
- use **MSE** when large mistakes deserve strong penalty;
- use **silhouette score** when you want to judge separation without true labels;
- use **adjusted Rand score** when you have known reference groups.

Section summary. Evaluation is not an afterthought. Metrics are how you translate model output into engineering meaning. A good model is not simply one with a large score, but one with the right score for the actual decision problem.

9 Model Inspection and Interpretability

9.1 9.1 Why inspection matters in engineering

Engineering decisions often require explanation, not only prediction. A strong numerical score is useful, but engineers also want to know which inputs are influential, whether the model behaves sensibly, and whether the learned relationship matches domain understanding. Scikit-Learn includes several inspection tools to support this process [8].

Inspection is especially important in safety-related work. A model that performs well for the wrong reasons can still be dangerous.

9.2 9.2 Permutation importance

9.2.1 Intuition

Permutation importance asks a simple question: if we randomly shuffle one feature after the model has been trained, how much does model performance get worse? If the score drops a lot, that feature was important. If the score barely changes, the feature was not contributing much [8][19].

9.2.2 Purpose

- understand feature influence after fitting;
- compare relative importance across features;
- identify features that may be weak or redundant.

9.2.3 Inputs and outputs

- **Inputs:** a fitted model, feature matrix X , target y , and a scoring rule.
- **Output:** an object containing mean importance values and variability across repeated shuffles.

9.2.4 A simple mathematical view

If $s(\hat{f}, X, y)$ is the model score before permutation and $\pi_j^{(r)}(X)$ means that feature j has been shuffled in repeat r , then a simple form of permutation importance is

$$I_j = s(\hat{f}, X, y) - \frac{1}{R} \sum_{r=1}^R s(\hat{f}, \pi_j^{(r)}(X), y)$$

A larger value of I_j means the score drops more when that feature is shuffled, so the feature is more important to the fitted model.

9.2.5 Clean example

```
import numpy as np
from sklearn.datasets import make_regression
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

X, y = make_regression(
    n_samples=300,
    n_features=5,
    n_informative=3,
    noise=8.0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = RandomForestRegressor(random_state=42)
model.fit(X_train, y_train)

importance = permutation_importance(
    model,
```

```

X_test,
y_test,
n_repeats=20,
random_state=42,
)

print("Mean importances:", importance.importances_mean)
print("Standard deviations:", importance.importances_std)

```

9.2.6 Mechanical Engineering interpretation

In a material-strength model, permutation importance can help answer questions such as: does curing age matter more than water content in this fitted model? In a structural monitoring model, it can show whether certain vibration features are driving the predictions.

9.2.7 Limitation

Permutation importance can be misleading when features are strongly correlated. If two inputs contain almost the same information, shuffling one may not hurt performance much because the other still carries that information [8][11].

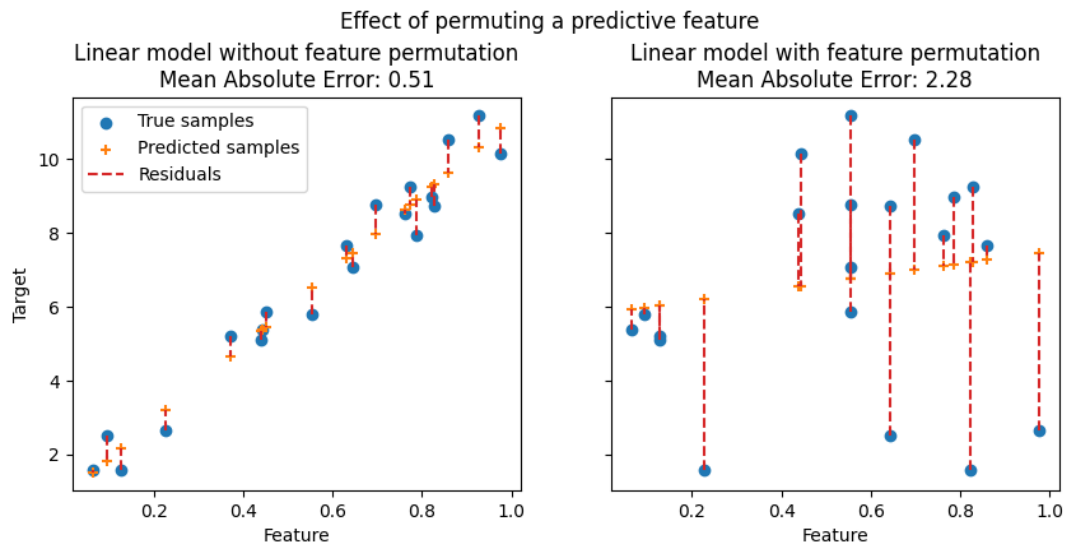


Figure 2. Permuting a predictive feature causes a large increase in error, which is the intuition behind permutation importance. Source: Scikit-Learn developers, permutation feature importance guide [19].

9.3 9.3 Partial dependence plots

9.3.1 Intuition

A partial dependence plot (PDP) shows how the model's average prediction changes as one feature changes, while averaging over the other features [8][17]. It helps answer questions such as: if moisture increases, does predicted deflection tend to increase or decrease?

9.3.2 Purpose

- visualize average model response to one or two features;
- detect linear, nonlinear, threshold-like, or saturation behavior;
- communicate model behavior to engineering audiences.

9.3.3 Inputs and outputs

- **Inputs:** a fitted estimator, data, and the feature index or name to inspect.
- **Output:** a display object that draws the partial dependence curve or surface.

9.3.4 A simple mathematical view

For one feature x_j , the partial dependence at a chosen value z can be written as

$$PD_j(z) = \frac{1}{n} \sum_{i=1}^n \hat{f}(z, x_{i,-j})$$

Here, $x_{i,-j}$ means all the other features of sample i , while the feature of interest is fixed at z . In plain language, the model prediction is recomputed many times with the chosen feature set to z , and then those predictions are averaged.

9.3.5 Clean example

```
from sklearn.datasets import make_regression
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import PartialDependenceDisplay

X, y = make_regression(
    n_samples=250,
    n_features=4,
    n_informative=3,
    noise=6.0,
    random_state=42,
)

model = RandomForestRegressor(random_state=42)
model.fit(X, y)

# Plot the average effect of features 0 and 1.
PartialDependenceDisplay.from_estimator(
    model,
    X,
    features=[0, 1],
)
```

9.3.6 Mechanical Engineering interpretation

If feature 0 represented water-cement ratio, a rising partial dependence curve would mean that, on average, increasing that feature increases the model's prediction. The exact physical interpretation depends on the dataset and the feature definition. The plot is therefore a model-behavior summary, not a law of mechanics.

9.3.7 Limitation

Partial dependence can be hard to interpret when inspected features are strongly correlated with other features. The official Scikit-Learn user guide explicitly notes that this can create unrealistic combinations of values during the averaging process [8].

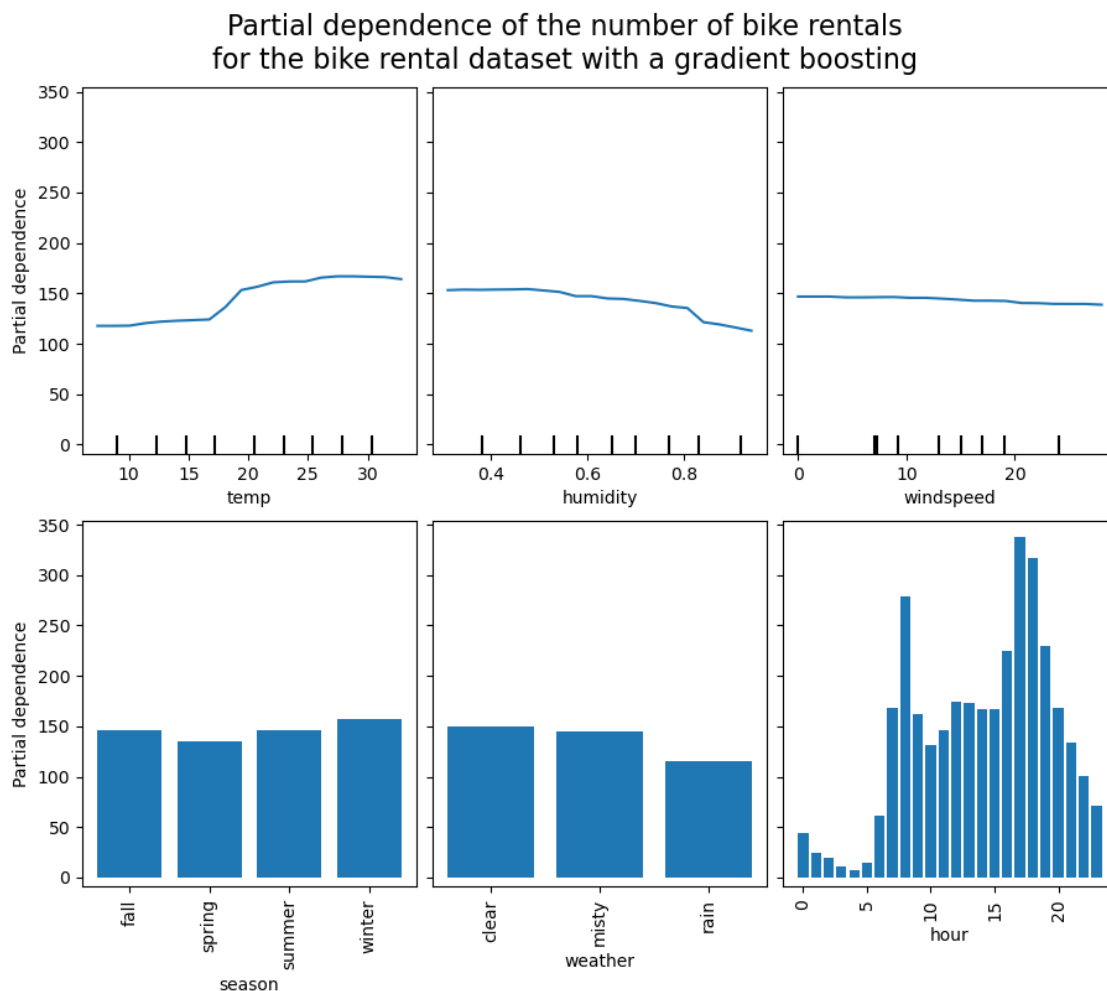


Figure 3. Example partial dependence displays showing how average model response changes with several features. Source: Scikit-Learn developers, “Advanced Plotting With Partial Dependence” example [17].

9.4 9.4 What inspection can and cannot tell you

Inspection tools are powerful, but they do not automatically prove causation.

Inspection **can** help you:

- understand which features the fitted model relies on;
- see whether the model responds smoothly or abruptly to an input;
- check whether model behavior roughly matches domain expectations.

Inspection **cannot** guarantee that:

- the relationship is causal;
- the model is correct outside the data range used for training;
- a visually smooth plot implies physical truth;
- an important feature is the only meaningful feature.

Section summary. Inspection connects prediction to explanation. In engineering, that mechanical component is essential, because models are most useful when their behavior can be discussed in the language of the physical problem.

10 Supervised Learning Models

10.1 10.1 Linear model family

10.1.1 Intuition

Linear models start from a simple idea: combine features using learned weights and form a prediction from that weighted sum. They are often the best place to begin because they are fast, interpretable, and provide strong baseline performance [3][9][10].

10.1.2 Family overview

Model	Typical task	Main idea	Strength	Limitation
LinearRegression	Regression	Straight-line relationship in feature space	Easy baseline, interpretable	Cannot capture strong nonlinearity by itself
LogisticRegression	Classification	Linear score passed through a probability function	Strong baseline classifier	Linear decision boundary unless features are engineered
Ridge	Regression	Linear regression with L_2 penalty	Handles correlated features better	Does not perform feature selection
Lasso	Regression	Linear regression with L_1 penalty	Can shrink some coefficients to zero	May be unstable with strongly correlated features
ElasticNet	Regression	Mix of L_1 and L_2 penalties	Balances ridge and lasso behavior	Needs more tuning choices

10.1.3 Inputs and outputs

For all these models, `fit(X, y)` learns coefficients from the feature matrix `X` and target `y`.

- In regression, `predict(X_new)` returns continuous values.
- In logistic regression, `predict(X_new)` returns class labels and `predict_proba(X_new)` returns class probabilities.

10.1.4 Worked example, four related regression models

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_absolute_error

X = np.array([
    [320, 180, 7, 0.45], [320, 180, 28, 0.45],
    [340, 175, 14, 0.42], [340, 175, 28, 0.42],
    [360, 170, 14, 0.40], [360, 170, 28, 0.40],
    [300, 190, 7, 0.50], [300, 190, 28, 0.50],
    [380, 165, 14, 0.38], [380, 165, 28, 0.38],
    [330, 178, 14, 0.44], [350, 172, 21, 0.41],
], dtype=float)

y = np.array([28.5, 37.8, 32.4, 40.9, 35.6, 44.1,
              24.7, 34.2, 38.8, 46.8, 33.7, 39.5], dtype=float)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

models = {
    "LinearRegression": LinearRegression(),
    "Ridge": make_pipeline(StandardScaler(), Ridge(alpha=1.0)),
    "Lasso": make_pipeline(StandardScaler(), Lasso(alpha=0.1, random_state=42)),
    "ElasticNet": make_pipeline(
        StandardScaler(),
        ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=42)
    ),
}

for name, model in models.items():
    model.fit(X_train, y_train)
```



```
y_pred = model.predict(X_test)
print(name, "MAE:", mean_absolute_error(y_test, y_pred))
```

In Scikit-Learn, logistic regression uses regularization by default, which is one reason scaling often helps.

10.1.5 Worked example, logistic regression for binary classification

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Example features: plasticity index, water content, dry density, fines content.
X = np.array([
    [18.0, 12.5, 1.89, 16.0], [20.5, 14.1, 1.84, 19.0],
    [11.0, 8.2, 2.02, 9.0], [24.0, 17.2, 1.75, 26.0],
    [13.5, 9.6, 1.98, 11.5], [26.0, 18.8, 1.70, 28.0],
    [12.4, 8.9, 2.05, 10.0], [22.8, 15.4, 1.80, 23.0],
    [10.6, 7.9, 2.06, 8.0], [25.3, 17.9, 1.73, 27.0],
], dtype=float)

y = np.array([0, 0, 0, 1, 0, 1, 0, 1, 0, 1], dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000, random_state=42)
)

model.fit(X_train, y_train)
print("Predicted classes:", model.predict(X_test))
print("Positive-class probabilities:", model.predict_proba(X_test)[:, 1])
```

10.1.6 Mechanical Engineering interpretation

- LinearRegression is a natural first model for strength, deflection, or deformation prediction.
- LogisticRegression is a strong baseline for binary screening, such as acceptable versus unacceptable condition.
- Ridge, Lasso, and ElasticNet are especially useful when you have many related features and want to control model complexity.

10.2 10.2 Support vector machines

10.2.1 Intuition

Support vector machines try to separate classes or fit regression functions using geometric margins. They are powerful when the boundary is not well described by a simple straight line, especially after using kernels [3][9].

10.2.2 Family overview

Model	Task	Output	Strength	Limitation
SVC	Classification	Class labels, and probabilities if enabled	Strong for complex boundaries	Sensitive to scaling and parameter choice
SVR	Regression	Continuous predictions	Flexible nonlinear regression	Slower on large datasets

10.2.3 Clean code example, support vector classifier

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_classification(
    n_samples=300,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", probability=True, random_state=42)
)

model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

10.2.4 Clean code example, support vector regressor

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

X, y = make_regression(
    n_samples=250,
    n_features=4,
    n_informative=3,
    noise=10.0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    SVR(kernel="rbf", C=3.0, epsilon=0.2)
)

model.fit(X_train, y_train)
print("R^2:", model.score(X_test, y_test))
```

10.2.5 Mechanical Engineering interpretation

SVC can be useful for classifying material types, structural condition states, or water-quality classes when decision boundaries are not simple. SVR can be useful for nonlinear prediction problems such as deformation or strength when a straight-line model is too limited.

10.3 10.3 Naive Bayes

10.3.1 Intuition

Naive Bayes models use Bayes' rule and a strong simplifying assumption: features are treated as conditionally independent given the class. The assumption is often not exactly true, but the models can still work surprisingly well [3][9].

10.3.2 Family overview

Model	Typical feature type	Common use
GaussianNB	Continuous features with roughly Gaussian behavior	Sensor and laboratory measurements

Model	Typical feature type	Common use
MultinomialNB	Counts or non-negative totals	Document term counts, event counts
BernoulliNB	Binary features	Yes or no inspection indicators

10.3.3 Gaussian Naive Bayes example

```

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

X, y = make_classification(
    n_samples=250,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))

```

10.3.4 Multinomial Naive Bayes, conceptual code template

```

from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB(alpha=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

```

Use MultinomialNB when features are counts or non-negative totals, not ordinary continuous laboratory measurements.

10.3.5 Bernoulli Naive Bayes, conceptual code template

```

from sklearn.naive_bayes import BernoulliNB

model = BernoulliNB(alpha=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

```

Use BernoulliNB when each feature records a yes or no condition, such as crack detected or not detected.

10.3.6 Mechanical Engineering interpretation

Gaussian Naive Bayes can act as a fast baseline for classification with continuous test results. Multinomial and Bernoulli variants are more common when features represent counts or binary checklists rather than ordinary continuous measurements.

10.4 10.4 Decision trees

10.4.1 Intuition

Decision trees split the feature space into regions using a sequence of if-then rules. They are often easy to explain because they resemble a chain of decisions [3][9].

10.4.2 Family overview

Model	Task	Output	Strength	Limitation
DecisionTreeClassifier	Classification	Class labels or probabilities	Easy to interpret	Can overfit badly
DecisionTreeRegressor	Regression	Continuous predictions	Handles nonlinear patterns	Can produce unstable trees

10.4.3 Decision tree classifier example

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(
    n_samples=250,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

10.4.4 Decision tree regressor example

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

X, y = make_regression(
    n_samples=250,
    n_features=4,
    n_informative=3,
    noise=10.0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = DecisionTreeRegressor(max_depth=4, random_state=42)
model.fit(X_train, y_train)
print("R^2:", model.score(X_test, y_test))
```

10.4.5 Mechanical Engineering interpretation

Decision trees can be useful when engineers want rule-like explanations such as “if moisture is high and dry density is low, classify as weak substrate.” They are also strong building blocks for larger ensemble models.

10.5 10.5 Ensemble learning

10.5.1 Intuition

Ensemble learning combines many simple learners to produce a stronger overall model. The key idea is that a collection of models can often be more stable and more accurate than a single model [3][9][10].

10.5.2 Family comparison

Model family	Main idea	Typical strength	Typical limitation
Random forest	Average many trees built from bootstrapped samples	Strong general-purpose baseline	Less transparent than a single tree
Gradient boosting	Add trees sequentially to correct previous errors	High predictive power	Needs careful tuning
AdaBoost	Focus new learners on previous mistakes	Can improve weak learners	Sensitive to noisy labels
Extra trees	Use stronger randomization in tree splits	Fast and often effective	May be less interpretable
Bagging	Fit many models on resampled	Reduces variance	Improvement depends on

Model family	Main idea	Typical strength	Typical limitation
	datasets and aggregate		base estimator

10.5.3 Random forest classifier example

```

from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=300,
    n_features=6,
    n_informative=4,
    n_redundant=0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))

```

10.5.4 Random forest regressor example

```

from sklearn.datasets import make_regression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

X, y = make_regression(
    n_samples=300,
    n_features=6,
    n_informative=4,
    noise=12.0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
print("R^2:", model.score(X_test, y_test))

```

10.5.5 Gradient boosting, conceptual code templates

The following short templates assume that `X_train`, `X_test`, `y_train`, and `y_test` already exist from a standard train and test split.

```
from sklearn.ensemble import GradientBoostingClassifier, GradientBoostingRegressor

clf = GradientBoostingClassifier(random_state=42)
reg = GradientBoostingRegressor(random_state=42)

clf.fit(X_train, y_train)
reg.fit(X_train, y_train)
```

Gradient boosting often performs very well on structured tabular data and is widely used in engineering prediction tasks.

10.5.6 AdaBoost, conceptual code templates

Again, assume that the training arrays already exist.

```
from sklearn.ensemble import AdaBoostClassifier, AdaBoostRegressor

clf = AdaBoostClassifier(random_state=42)
reg = AdaBoostRegressor(random_state=42)

clf.fit(X_train, y_train)
reg.fit(X_train, y_train)
```

10.5.7 Extra trees, conceptual code templates

Again, assume that the training arrays already exist.

```
from sklearn.ensemble import ExtraTreesClassifier, ExtraTreesRegressor

clf = ExtraTreesClassifier(n_estimators=200, random_state=42)
reg = ExtraTreesRegressor(n_estimators=200, random_state=42)

clf.fit(X_train, y_train)
reg.fit(X_train, y_train)
```

10.5.8 Bagging, conceptual code templates

Again, assume that the training arrays already exist.

```
from sklearn.ensemble import BaggingClassifier, BaggingRegressor
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

clf = BaggingClassifier(
```



```

estimator=DecisionTreeClassifier(max_depth=4),
n_estimators=100,
random_state=42,
)

reg = BaggingRegressor(
    estimator=DecisionTreeRegressor(max_depth=4),
    n_estimators=100,
    random_state=42,
)

clf.fit(X_train, y_train)
reg.fit(X_train, y_train)

```

10.5.9 Mechanical Engineering interpretation

- Random forests are strong baseline models for condition classification, material-property prediction, and many tabular engineering datasets.
- Gradient boosting can be very effective when the pattern is nonlinear but structured.
- AdaBoost, extra trees, and bagging are useful alternative ensemble strategies, especially when you want to compare different bias-variance trade-offs.

Section summary. Supervised learning in Scikit-Learn includes simple linear baselines, flexible geometric models, probability-based classifiers, rule-based trees, and powerful ensembles. The right choice depends on the problem structure, data size, need for interpretability, and cost of error.

11 Mathematical Background for Supervised Models

11.1 11.1 Linear regression

Linear regression assumes the target can be approximated by a weighted sum of the features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The coefficients β_j are chosen to reduce the squared prediction errors. In plain language, the model learns how much each feature contributes to the prediction, assuming the effect is additive and linear in the feature space.

For material strength prediction, a positive coefficient for curing age would mean that, all else being equal, larger curing age tends to increase predicted strength.

11.2 11.2 Logistic regression

Logistic regression begins with a linear score,

$$\eta = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

and converts it into a probability using the logistic function:

$$P(y=1 \mid x) = \frac{1}{1+e^{-\eta}}$$

This keeps the output between 0 and 1. A threshold, often 0.5, then turns the probability into a class label.

11.3 Ridge, lasso, and elastic net regularization

Regularization adds a penalty to the objective so the model does not use unnecessarily large coefficients.

11.3.1 Ridge regression

$$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_j \beta_j^2$$

The L_2 penalty, $\sum_j \beta_j^2$, shrinks coefficients toward zero. It is useful when features are correlated.

11.3.2 Lasso regression

$$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_j |\beta_j|$$

The L_1 penalty can drive some coefficients exactly to zero, which makes lasso useful for feature selection.

11.3.3 Elastic net

Elastic net combines both penalties:

$$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \alpha \left[(1-r) \sum_j \beta_j^2 + r \sum_j |\beta_j| \right]$$

where r is the mixing ratio, usually called `l1_ratio` in Scikit-Learn.

11.4 Support vector machines

For classification, the support vector machine looks for a separating boundary with the largest possible margin between classes. The points closest to the boundary are called **support vectors** because they determine the position of that boundary.

The kernel trick allows the model to behave as if it had transformed the data into a richer feature space without explicitly computing that space. This is why nonlinear SVMs can model curved decision boundaries.

For regression, support vector regression tries to keep predictions within an ϵ -wide error tube around the data while keeping the function as flat as possible.

11.5 Naive Bayes classification

Naive Bayes uses Bayes' rule:

$$P(C | x) = \frac{P(x | C)P(C)}{P(x)}$$

The model then makes the simplifying assumption that features are conditionally independent given the class. Under that assumption,

$$P(x | C) = \prod_{j=1}^p P(x_j | C)$$

This assumption is often unrealistic, but the resulting models are simple, fast, and effective in many settings.

11.6 11.6 Decision trees

Decision trees recursively split the feature space into regions. In classification, the algorithm searches for splits that reduce impurity, often measured by quantities such as Gini impurity or entropy. In regression, the algorithm looks for splits that reduce target variance inside the child regions.

A tree prediction is then the class label or mean target value associated with the terminal region into which the sample falls.

11.7 11.7 Random forests and ensemble learning

A random forest trains many decision trees on bootstrapped samples and averages their predictions or takes a majority vote. The main idea is variance reduction. Individual trees can be noisy and unstable. Averaging many decorrelated trees makes the result more stable.

More broadly, ensemble learning combines multiple learners to improve performance. Bagging reduces variance by averaging. Boosting reduces bias by adding learners sequentially to correct previous mistakes.

11.8 11.8 A gentle engineering takeaway

The mathematics may look different across models, but there are recurring themes.

- Linear models learn coefficients.
- Logistic regression turns a linear score into a probability.
- Regularization controls complexity.
- SVMs use geometric margins.
- Naive Bayes uses probabilities and Bayes' rule.
- Trees split the feature space into regions.
- Ensembles combine many learners for stability or accuracy.

Section summary. The mathematics of supervised learning becomes easier once you see each model as an answer to one question: how should the model convert measured inputs into a prediction while controlling error and complexity?

12 Unsupervised Learning Models

12.1 12.1 Clustering algorithms

12.1.1 Intuition

Clustering looks for natural groupings inside the data when no target labels are provided. In Mechanical Engineering, clustering can be useful for grouping materials, projects, mounting conditions, or production throughput patterns that behave similarly.

12.1.2 Family overview

Model	Main idea	Strength	Limitation
KMeans	Group points around centroids	Fast and easy to understand	Assumes roughly compact clusters
AgglomerativeClustering	Merge nearby groups step by step	Reveals hierarchical structure	Can be sensitive to linkage choice
DBSCAN	Find dense regions and label sparse points as noise	Good for irregular shapes and outliers	Sensitive to density settings
SpectralClustering	Use a similarity graph before clustering	Good for complex shapes	More computationally expensive
MeanShift	Move points toward dense regions	Finds cluster centers automatically in some cases	Can be slow
AffinityPropagation	Pass messages among samples to choose exemplars	Does not require choosing cluster count directly	Can be expensive and sensitive to preferences

12.1.3 K-Means example

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X = np.array([
    [1.0, 1.1], [1.2, 0.9], [0.9, 1.0],
    [5.0, 5.2], [5.3, 4.8], [4.8, 5.1],
    [8.9, 1.1], [9.2, 0.8], [8.7, 1.0],
], dtype=float)

model = KMeans(n_clusters=3, n_init="auto", random_state=42)
labels = model.fit_predict(X)
```

```
print("Cluster labels:", labels)
print("Silhouette score:", silhouette_score(X, labels))
```

12.1.4 Agglomerative clustering example

```
from sklearn.cluster import AgglomerativeClustering

model = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = model.fit_predict(X)
print(labels)
```

12.1.5 DBSCAN example

```
from sklearn.cluster import DBSCAN

model = DBSCAN(eps=0.6, min_samples=3)
labels = model.fit_predict(X)
print(labels)
```

In DBSCAN, samples labeled -1 are treated as noise or outliers.

12.1.6 Spectral clustering, conceptual code template

```
from sklearn.cluster import SpectralClustering

model = SpectralClustering(n_clusters=3, random_state=42)
labels = model.fit_predict(X)
```

12.1.7 Mean shift, conceptual code template

```
from sklearn.cluster import MeanShift

model = MeanShift()
labels = model.fit_predict(X)
```

12.1.8 Affinity propagation, conceptual code template

```
from sklearn.cluster import AffinityPropagation

model = AffinityPropagation(random_state=42)
labels = model.fit_predict(X)
```

12.1.9 Mechanical Engineering interpretation

- K-means can group materials or site observations into broad categories.
- Agglomerative clustering is useful when a hierarchy matters, such as grouping projects into larger families.
- DBSCAN is attractive when abnormal or isolated observations may be meaningful rather than just noise.

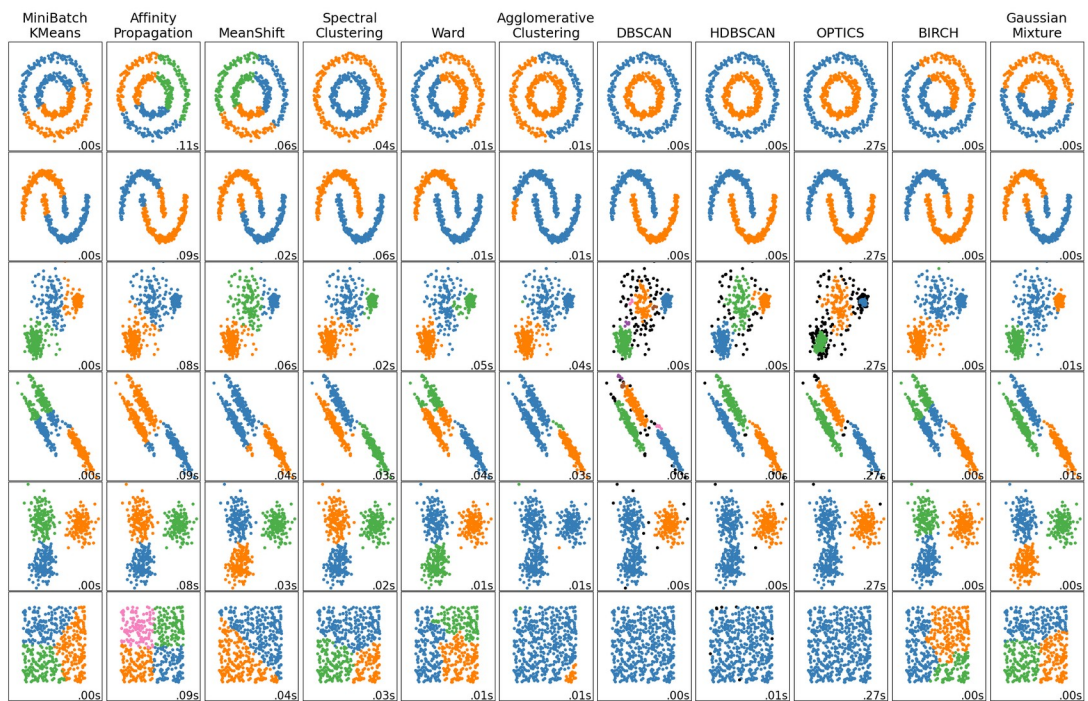


Figure 4. Comparison of several clustering algorithms on toy datasets. Source: Scikit-Learn developers, clustering comparison example [15].

12.2 12.2 Dimensionality reduction

12.2.1 Intuition

Dimensionality reduction compresses many variables into a smaller set of informative variables. This can help with visualization, noise reduction, and preprocessing for later analysis.

12.2.2 Family overview

Model	Type	Main use
PCA	Linear unsupervised	Capture directions of largest variance
TSNE	Nonlinear unsupervised	Visualize local neighborhood structure in 2D or 3D
LinearDiscriminantAnalysis	Supervised dimensionality reduction	Separate known classes while reducing dimension
NMF	Matrix factorization with non-negativity	Parts-based representation for non-negative data
Isomap	Nonlinear manifold learning	Preserve neighborhood geodesic structure

12.2.3 Principal component analysis example

```
from sklearn.datasets import make_regression
from sklearn.pipeline import make_pipeline
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, _ = make_regression(
    n_samples=200,
    n_features=6,
    n_informative=4,
    random_state=42,
)

model = make_pipeline(StandardScaler(), PCA(n_components=2))
X_reduced = model.fit_transform(X)
print("Original shape:", X.shape)
print("Reduced shape:", X_reduced.shape)

```

12.2.4 t-SNE example

The next example reuses the same feature matrix X from the PCA example above.

```

from sklearn.manifold import TSNE

model = TSNE(n_components=2, random_state=42)
X_embedded = model.fit_transform(X)
print(X_embedded.shape)

```

t-SNE is mainly a visualization tool. It is excellent for exploring structure, but it is not usually the first choice for routine predictive preprocessing.

12.2.5 Linear discriminant analysis in its dimensionality-reduction role

LinearDiscriminantAnalysis, often abbreviated LDA, is sometimes introduced as a classifier. It can also be used for dimensionality reduction when class labels are known. This means it is **not fully unsupervised**, even though it appears in some dimensionality-reduction lists. It chooses directions that separate known classes as well as possible.

```

from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X_lda_input, y_lda = make_classification(
    n_samples=150,
    n_features=6,
    n_informative=4,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)

```

```
)

lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_lda_input, y_lda)
print(X_lda.shape)
```

12.2.6 Non-negative matrix factorization example

This example also reuses the same feature matrix X from the PCA example, after converting it to non-negative values.

```
import numpy as np
from sklearn.decomposition import NMF

X_nonnegative = np.abs(X) # NMF requires non-negative data.
model = NMF(n_components=2, init="nndsvda", random_state=42)
W = model.fit_transform(X_nonnegative)
H = model.components_
print(W.shape, H.shape)
```

12.2.7 Isomap example

This example reuses the same feature matrix X from the PCA example above.

```
from sklearn.manifold import Isomap

model = Isomap(n_components=2, n_neighbors=5)
X_iso = model.fit_transform(X)
print(X_iso.shape)
```

12.2.8 Mechanical Engineering interpretation

- PCA can compress many structural monitoring features into a few principal components.
- t-SNE can visualize high-dimensional monitoring or materials datasets.
- LDA can help separate known material classes or damage states in a lower-dimensional representation.
- NMF is useful when features are naturally non-negative, such as counts, intensities, or spectra.
- Isomap can reveal curved low-dimensional structure inside nonlinear datasets.

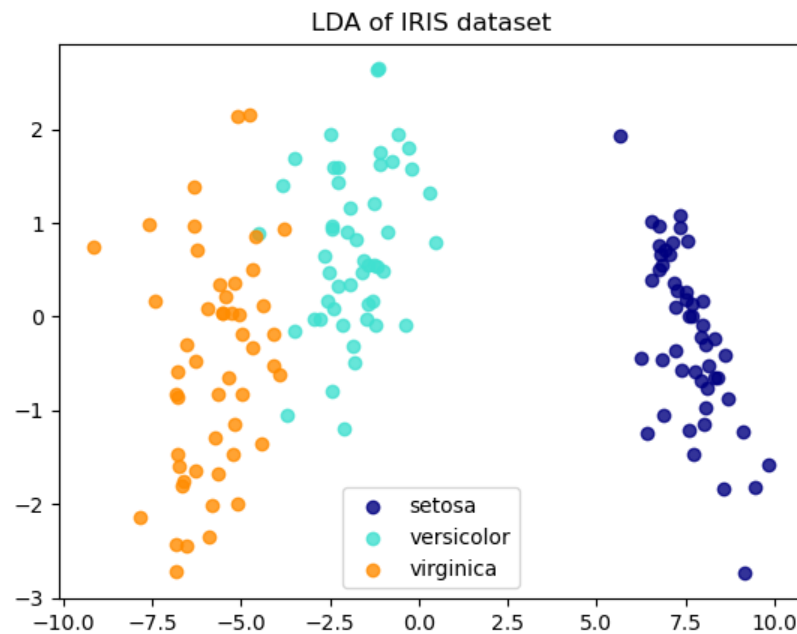


Figure 5. Comparison of principal component analysis and linear discriminant analysis on the Iris dataset. Source: Scikit-Learn developers, PCA versus LDA example [16].

12.3 12.3 Anomaly detection

12.3.1 Intuition

Anomaly detection looks for unusual samples that do not resemble the majority. In Mechanical Engineering, anomalies may correspond to unusual sensor behavior, potential damage, rare thermal-fluid events, or abnormal production throughput patterns.

12.3.2 One-class SVM

OneClassSVM learns a boundary around normal data. Samples inside that region are treated as inliers and samples outside it are treated as anomalies.

```
import numpy as np
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(42)
X_train_normal = rng.normal(loc=[0.0, 0.0], scale=[0.6, 0.6], size=(200, 2))
X_test = np.vstack([
    rng.normal(loc=[0.0, 0.0], scale=[0.6, 0.6], size=(20, 2)),
    rng.uniform(low=-4.0, high=4.0, size=(10, 2)),
])

model = OneClassSVM(kernel="rbf", gamma="scale", nu=0.05)
```

```
model.fit(X_train_normal)
labels = model.predict(X_test)

# 1 means inlier, -1 means anomaly.
print(labels[:10])
```

12.3.3 Isolation forest

IsolationForest isolates anomalies by repeatedly splitting the data with random trees. Unusual points tend to be isolated in fewer splits, so they receive more anomalous scores. The next example reuses `X_train_normal` and `X_test` from the one-class SVM example above.

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(contamination=0.05, random_state=42)
model.fit(X_train_normal)
labels = model.predict(X_test)

# 1 means inlier, -1 means anomaly.
print(labels[:10])
```

12.3.4 Preparing normal-only training data

For anomaly detection, it is common to train mainly on data believed to represent normal behavior. In structural health monitoring, for example, a model may be fit on vibration features recorded during healthy operation and later used to flag abnormal time windows.

12.3.5 Mechanical Engineering interpretation

- OneClassSVM can identify unusual feature patterns in structural monitoring data.
- IsolationForest can flag abnormal laboratory results, sensor windows, or production throughput states without needing many labeled anomaly examples.

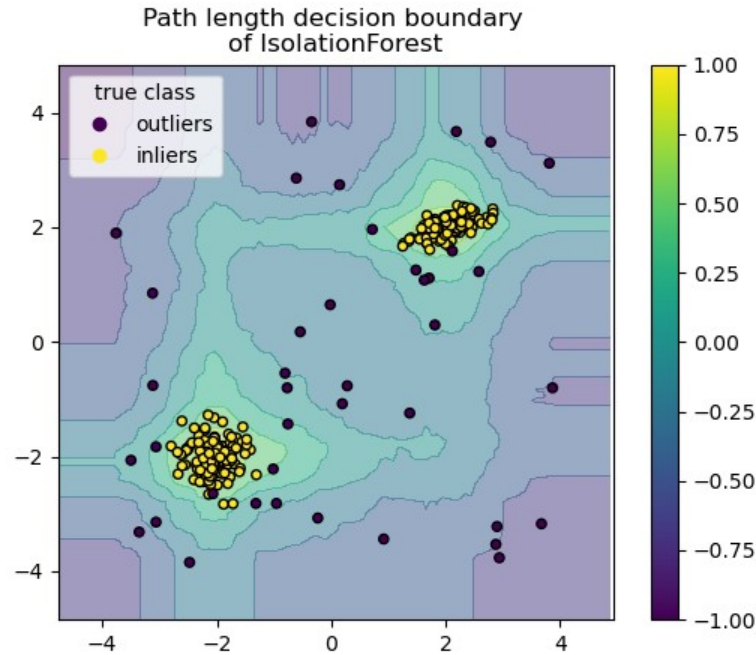


Figure 6. Isolation Forest example illustrating regions that are treated as normal and regions treated as anomalous. Source: Scikit-Learn developers, Isolation Forest example [18].

Section summary. Unsupervised learning helps you discover structure without target labels. In practice, clustering groups similar cases, dimensionality reduction compresses complex data, and anomaly detection highlights unusual behavior.

13 Mathematical Background for Unsupervised Models

13.1 13.1 Clustering ideas in simple terms

Clustering tries to maximize similarity inside groups and minimize similarity between groups. The exact meaning of similarity depends on the method.

- K-means uses distances to centroids.
- Hierarchical clustering uses linkage rules between groups.
- DBSCAN uses local density.
- Spectral methods use a graph-based similarity structure.

13.2 13.2 Centroid-based clustering and k-means

K-means chooses cluster centers, called centroids, and assigns each point to the nearest one. It then updates the centroids and repeats.

The objective is to reduce the within-cluster sum of squared distances:

$$\sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid of cluster k and C_k is the set of points assigned to that cluster.

13.3 Hierarchical clustering

Agglomerative clustering starts with each point as its own cluster and then repeatedly merges the closest clusters. The meaning of “closest” depends on the linkage rule.

- **Single linkage** uses the closest pair of points.
- **Complete linkage** uses the farthest pair of points.
- **Average linkage** uses average distance.
- **Ward linkage** merges clusters to minimize the increase in within-cluster variance.

This produces a hierarchy rather than only one partition.

13.4 Density-based clustering

DBSCAN treats clusters as dense regions separated by sparser regions. Two settings matter:

- `eps`, the neighborhood radius;
- `min_samples`, the minimum number of points needed to treat a region as dense.

This allows DBSCAN to detect irregular cluster shapes and label sparse points as noise.

13.5 Dimensionality reduction concepts

When there are many features, some may overlap in information. Dimensionality reduction searches for a smaller representation that keeps the most useful structure.

13.5.1 Principal component analysis and variance

PCA finds orthogonal directions, called principal components, that capture as much variance as possible. The first principal component captures the largest variance, the second captures the next largest variance subject to being orthogonal to the first, and so on.

In simple language, PCA rotates the coordinate system to align with the most informative directions in the data.

13.5.2 t-SNE and Isomap, neighborhood intuition

Both t-SNE and Isomap are neighborhood-based nonlinear methods, but they serve different purposes.

- **t-SNE** tries to place nearby points close together in the low-dimensional map, making it excellent for visualization of local neighborhoods.

- **Isomap** tries to preserve geodesic relationships along the data manifold, which can reveal curved low-dimensional structure.

13.6 13.6 Linear discriminant analysis as dimensionality reduction

LDA uses known class labels and finds directions that separate classes while keeping within-class spread small. It therefore belongs between supervised classification and dimensionality reduction. It is included in many dimensionality-reduction discussions, but conceptually it is supervised.

13.7 13.7 Anomaly detection intuition

13.7.1 One-class SVM

One-class SVM learns a boundary around normal data. Points outside the learned region are treated as unusual.

13.7.2 Isolation forest

Isolation Forest relies on a different idea. If a point is very unusual, a random splitting procedure isolates it quickly. So anomalies tend to have shorter average path lengths in the random trees.

13.8 13.8 Silhouette score intuition revisited

The silhouette score compares how close a point is to its own cluster versus the nearest alternative cluster. If a point is much closer to its own group than to others, the score is high. If it is equally close to multiple groups, the score is near zero. If it seems closer to another group, the score can become negative.

13.9 13.9 Adjusted Rand score intuition revisited

The adjusted Rand score compares two labelings by checking which pairs of samples are placed together or apart in each labeling. It then adjusts the agreement to account for what might happen by chance.

Section summary. The mathematics of unsupervised learning is really the mathematics of structure: distances, densities, neighborhoods, variance, and unusualness.

14 Engineering Interpretation and Good Practice

14.1 14.1 Why interpretation matters more in engineering than in many toy examples

In classroom examples, a model is often judged by a score alone. In engineering, that is not enough. You also need to ask whether the inputs are physically meaningful, whether the training data represent the intended application range, whether the errors are acceptable in engineering units, and whether the model would still behave sensibly when conditions change.

14.2 14.2 Model choice guide for common Mechanical Engineering problems

Mechanical Engineering task	Likely problem type	Good first models to try	Why
Mechanical material strength prediction	Regression	Linear regression, ridge, random forest regressor, gradient boosting regressor	Start simple, then allow nonlinearity
Material or substrate classification	Classification	Logistic regression, SVC, random forest classifier	Strong tabular-data baselines
Machined-surface defect classification	Classification	Logistic regression, decision tree, random forest, gradient boosting	Clear baseline to stronger tree ensembles
Deformation or deflection prediction	Regression	Linear regression, SVR, random forest regressor	Mix of simple and nonlinear options
Coolant quality class prediction	Classification	Logistic regression, GaussianNB, random forest classifier	Probabilities and interpretable baselines
Clustering materials or site observations	Clustering	K-means, agglomerative clustering, DBSCAN	Different views of similarity and density
Sensor-data compression	Dimensionality reduction	PCA, LDA if labels exist, Isomap	Noise reduction and visualization
Structural health monitoring anomaly detection	Anomaly detection	Isolation Forest, OneClassSVM	Useful when anomaly labels are scarce

14.3 14.3 A practical checklist before trusting a result

Ask the following questions.

1. Were training and testing kept separate?
2. Was scaling placed inside the pipeline when needed?
3. Were the chosen metrics appropriate for the engineering consequences?
4. Does the model behave sensibly when inspected?
5. Are the predictions being used inside the range of the training data?
6. Would a domain expert consider the main dthermal loops physically plausible?

14.4 14.4 Common beginner mistakes

- using accuracy alone for an imbalanced classification problem;
- scaling the full dataset before splitting;
- comparing models with different preprocessing choices in an unfair way;
- trusting a high R^2 without checking MAE in physical units;
- confusing explanation with causation;

- using t-SNE as if it were an ordinary predictive preprocessing step;
- forgetting that LDA used for dimensionality reduction still needs class labels;
- assuming anomaly detectors return a probability of damage rather than an inlier or outlier label.

14.5 14.5 Mechanical Engineering mini-cases

14.5.1 Mechanical material strength prediction

A sensible baseline sequence is: linear regression, ridge regression, then random forest regressor. Compare MAE in MPa, inspect feature importance, and check whether the dominant features match materials knowledge.

14.5.2 Material type or substrate classification

Start with logistic regression and a confusion matrix. If the classes are not linearly separable, compare SVC and random forest classifier. Recall may matter most if missing weak material is risky.

14.5.3 Structural health monitoring

Use PCA to reduce many vibration features, then compare a supervised classifier if labels exist, or Isolation Forest if anomaly labels are scarce. Use permutation importance or partial dependence only after confirming the model is reasonably accurate.

14.5.4 Coolant quality classification

Use classification metrics, especially precision and recall, because different mistakes can have different operational costs. Consider simple interpretable baselines before moving to complex models.

14.6 14.6 Final perspective

Scikit-Learn is best viewed as an engineering analysis tool, not an automatic decision-maker. Its outputs are most valuable when combined with physical reasoning, code transparency, careful evaluation, and engineering judgment.

Section summary. Good practice in engineering machine learning means careful workflow, appropriate metrics, sensible interpretation, and respect for the limits of the available data.

15 Version Check and Setup

15.1 15.1 Installing the library

Scikit-Learn 1.8.0 can be installed or upgraded using standard Python package tools. The official release highlights page shows commands such as the following [2].

```
pip install --upgrade scikit-learn
```

If you need the exact version for class consistency, you can pin it:

```
pip install scikit-learn==1.8.0
```

15.2 15.2 Checking the installed version

```
import sklearn  
print(sklearn.__version__)
```

15.3 15.3 Recommended companion libraries for this tutorial

```
pip install numpy matplotlib pandas
```

Pandas is not required for the examples in this guide, but many real engineering datasets arrive in tabular form, so it is often useful.

15.4 15.4 A beginner setup habit worth keeping

At the top of a notebook or script, print the Scikit-Learn version. This makes it easier to reproduce results later and reduces confusion when documentation changes across versions.

Section summary. Always know which version of the library you are using. Small differences in version can affect defaults, examples, and warnings.

16 References

1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
2. Scikit-Learn developers (2026). *Release Highlights for Scikit-Learn 1.8*. Available at: https://scikit-learn.org/stable/auto_examples/release_highlights/plot_release_highlights_1_8_0.html (accessed 28 March 2026).
3. Scikit-Learn developers (2026). *Scikit-Learn User Guide, stable version 1.8.0*. Available at: https://scikit-learn.org/stable/user_guide.html (accessed 28 March 2026).
4. Scikit-Learn developers (2026). *Model evaluation: quantifying the quality of predictions*. Available at: https://scikit-learn.org/stable/model_evaluation.html (accessed 28 March 2026).
5. Scikit-Learn developers (2026). *Preprocessing data*. Available at: <https://scikit-learn.org/stable/preprocessing.html> (accessed 28 March 2026).
6. Scikit-Learn developers (2026). *Pipelines and composite estimators*. Available at: <https://scikit-learn.org/stable/compose.html> (accessed 28 March 2026).
7. Scikit-Learn developers (2026). *Common pitfalls and recommended practices*. Available at: https://scikit-learn.org/stable/common_pitfalls.html (accessed 28 March 2026).

8. Scikit-Learn developers (2026). *Inspection*. Available at: <https://scikit-learn.org/stable/inspection.html> (accessed 28 March 2026).
9. Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). New York: Springer.
10. James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning: With Applications in Python*. New York: Springer.
11. Molnar, C. (2022). *Interpretable Machine Learning* (2nd ed.). Available at: <https://christophm.github.io/interpretable-ml-book/> (accessed 28 March 2026).
12. Bahadori-Jahromi, A., Room, S., Paknahad, C., Altekreeti, M., Tariq, Z., and Tahayori, H. (2025). The Role of Artificial Intelligence and Machine Learning in Advancing Mechanical Engineering: A Comprehensive Review. *Applied Sciences*, 15(19), 10499. <https://doi.org/10.3390/app151910499>
13. Gamil, Y. (2023). Machine learning in material technology: A review of current researches, trends, and applications. *Frontiers in Built Environment*, 9, 1145591. <https://doi.org/10.3389/fbuil.2023.1145591>
14. Scikit-Learn developers (2026). *Confusion matrix example*. Available at: https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html (accessed 28 March 2026).
15. Scikit-Learn developers (2026). *Comparing different clustering algorithms on toy datasets example*. Available at: https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html (accessed 28 March 2026).
16. Scikit-Learn developers (2026). *Comparison of LDA and PCA 2D projection of Iris dataset example*. Available at: https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_vs_lda.html (accessed 28 March 2026).
17. Scikit-Learn developers (2026). *Advanced Plotting With Partial Dependence example*. Available at: https://scikit-learn.org/stable/auto_examples/inspection/plot_partial_dependence.html (accessed 28 March 2026).
18. Scikit-Learn developers (2026). *IsolationForest example*. Available at: https://scikit-learn.org/stable/auto_examples/ensemble/plot_isolation_forest.html (accessed 28 March 2026).
19. Scikit-Learn developers (2026). *Permutation feature importance*. Available at: https://scikit-learn.org/stable/permutation_importance.html (accessed 28 March 2026).